Лабораторная работа № 9

Тема: Транзакции и целостность данных

Цель: На примере существующих товаров и категорий сравнить операции без транзакции и с транзакцией Prisma; самостоятельно реализовать создание заказа с позициями в одной транзакции.

Дисциплина: РО 4.3. Интеграция серверных приложений с реляционными базами данных

Лабораторная работа № 9. Транзакции и целостность данных

Тема: транзакции Prisma ORM 7 на базе shop_db. Учебный пример использует существующие таблицы categories и products. Самостоятельная часть: orders и order_items.

Цель: сравнить результаты нескольких операций с транзакцией и без неё; освоить COMMIT и ROLLBACK; самостоятельно определить границы транзакции при создании заказа интернет-магазина.

> **Безопасность данных**
>
> Работайте только с учебной базой shop_db. Существующие товары с id = 1 и 2 не изменяйте и не удаляйте. Не используйте migrate reset. Для опытов создаются отдельные записи с названием LAB9.

1. Исходные условия

Инструменты: Node.js, Prisma ORM 7, PostgreSQL, pgAdmin, готовый учебный проект. В прошлой лабораторной БД уже заполнена: две категории и два товара. Повторно создавать их не нужно.

Начальные данные (сверьте с вашей базой): categories: 1 — «Ноутбуки», 2 — «Аксессуары»; products: 1 — «Laptop Pro» (420000, category_id = 1), 2 — «Wireless Mouse» (12000, category_id = 2). Поля products: id, name, price, description, in_stock, category_id. Для опыта ошибку вызовем ссылкой на отсутствующую категорию 999999. Нужен внешний ключ products.category_id → categories.id (он уже настроен в учебной БД).

```
pgAdmin: посмотреть уже созданные категории и товары

SELECT * FROM categories;
SELECT * FROM products;
```

Убедитесь, что категория 1 («Ноутбуки») и категория 2 («Аксессуары») существуют, а товара с названием «LAB9 Кабель» и «LAB9 Чехол» ещё нет. Если в вашей БД их нет, используйте свои существующие категории и исправьте category_id в JavaScript. Не меняйте Laptop Pro и Wireless Mouse.

```
Простая проверка учебных товаров

SELECT * FROM products WHERE name = 'Laptop Pro';
SELECT * FROM products WHERE name = 'Wireless Mouse';
```

2. Подготовка тестового файла

Создайте в корне проекта файл transaction-demo.js. В предыдущих работах уже настроены Prisma Client и config/prisma.js, где экспортируется объект prisma. Запускайте файл из корня проекта.

> **Важно о запуске**
>
> Для Prisma 7 с сгенерированным клиентом на TypeScript используйте запуск через tsx, если он уже настроен в предыдущих лабораторных. При другом расположении config/prisma.js исправьте путь импорта.

3. Пример БЕЗ транзакции

Выполните две операции добавления товаров. Первая привязана к существующей категории 2 («Аксессуары»), вторая — к категории 999999, которой в нашей учебной базе нет. Здесь используются две отдельные операции без общей транзакции.

transaction-demo.js — вариант без транзакции

```
import { prisma } from "./config/prisma.js";

async function main() {
  try {
    // Категория 2 существует (Аксессуары).
    await prisma.products.create({
      data: {
        name: "LAB9 Кабель",
        price: 2500,
        description: "Учебный товар",
        in_stock: true,
        category_id: 2
      }
    });
    console.log("Первый товар добавлен");

    // Категории 999999 не существует: ожидаем ошибку FK.
    await prisma.products.create({
      data: {
        name: "LAB9 Чехол",
        price: 3900,
        description: "Учебный товар",
        in_stock: true,
        category_id: 999999
      }
    });
  } catch (error) {
    console.log("Ошибка:", error.code ?? error.message);
  } finally {
    await prisma.$disconnect();
  }
}

main();
```

Запуск примера

```
npx tsx transaction-demo.js
```

```
pgAdmin: проверить, что сохранилось
```

```sql
SELECT * FROM products;
```

Найдите в результате SELECT строки «LAB9 Кабель» и «LAB9 Чехол». Ожидается: «LAB9 Кабель» есть, «LAB9 Чехол» нет. Первый INSERT уже завершился отдельно, и ошибка второго INSERT не отменяет первый. Товары «Laptop Pro» и «Wireless Mouse» остаются без изменений.

4. Удаление тестовой записи

Перед следующим опытом удалите только тестовый товар «LAB9 Кабель». Оригинальные товары Laptop Pro и Wireless Mouse удалять нельзя. SQL-код ниже относится исключительно к учебной записи, созданной на предыдущем шаге.

```
Удаление только тестового товара
```

```sql
DELETE FROM products WHERE name = 'LAB9 Кабель';
SELECT * FROM products;
```

5. Тот же сценарий С транзакцией

Теперь те же два добавления выполняются внутри prisma.$transaction(async (tx) => { ... }). Важно: оба запроса идут через tx. Если второй запрос ошибётся, PostgreSQL отменит и первый INSERT. Для опыта файл transaction-demo.js замените следующим кодом.

```
transaction-demo.js — вариант с транзакцией
```

```js
import { prisma } from "./config/prisma.js";

async function main() {
  try {
    await prisma.$transaction(async (tx) => {
      // Операция 1 — категория существует.
      await tx.products.create({
        data: {
          name: "LAB9 Кабель",
          price: 2500,
          description: "Учебный товар",
          in_stock: true,
          category_id: 2
        }
      });

      // Операция 2 — категория отсутствует, будет ошибка.
      await tx.products.create({
        data: {
          name: "LAB9 Чехол",
          price: 3900,
          description: "Учебный товар",
          in_stock: true,
          category_id: 999999
        }
      });
    });
```

```
    console.log("COMMIT: оба товара добавлены");
  } catch (error) {
    console.log("ROLLBACK:", error.code ?? error.message);
  } finally {
    await prisma.$disconnect();
  }
}

main();
```

```
Запуск транзакции
npx tsx transaction-demo.js
```

```
pgAdmin: проверьте откат простым SELECT
SELECT * FROM products;
```

Ожидается: ни «LAB9 Кабель», ни «LAB9 Чехол» не появились. Оба добавления были частью одной транзакции, поэтому ошибка второй операции привела к ROLLBACK всей транзакции. Два исходных товара по-прежнему существуют.

6. Успешная транзакция (COMMIT)

Для успешного COMMIT измените в файле только одну строку: во втором tx.products.create замените category_id: 999999 на category_id: 1 (категория «Ноутбуки»). Теперь оба товара ссылаются на реальные категории: первый — 2, второй — 1. Повторите запуск.

```
Повторный запуск
npx tsx transaction-demo.js
```

```
pgAdmin: проверить успешное сохранение
SELECT * FROM products;
```

7. Сравнение результатов

Сценарий	LAB9 Кабель	LAB9 Чехол
Без транзакции; категория 999999	Сохранён	Не сохранён
С tx; категория 999999	Не сохранён	Не сохранён
С tx; категории 2 и 1	Сохранён	Сохранён

Проверяйте состояние в PostgreSQL

Сообщение console.log внутри callback не означает COMMIT. До успешного завершения всей транзакции данные могут быть отменены.

8. Очистка учебных данных

```
Удаление только учебных записей LAB9

DELETE FROM products WHERE name = 'LAB9 Кабель';
DELETE FROM products WHERE name = 'LAB9 Чехол';
SELECT * FROM products;
```

Убедитесь, что после удаления тестовых записей сохранились товары Laptop Pro (id = 1) и Wireless Mouse (id = 2). Если к тестовым товарам уже привязаны новые записи order_items, удаление может быть запрещено внешним ключом; в таком случае не удаляйте данные вслепую.

9. Самостоятельная работа: создание заказа магазина

Самостоятельное задание: разработать оформление заказа из двух товаров, которые уже есть в products: Laptop Pro (id = 1) и Wireless Mouse (id = 2). Нужно создать одну запись orders и две связанные записи order_items. Все действия, связанные с созданием заказа, должны быть внутри ОДНОЙ транзакции Prisma.

9.1. Какие данные нужно подготовить

1. Создайте таблицы orders (заказ) и order_items (позиции заказа). Таблицы categories и products уже существуют: их не пересоздавайте.
2. Предусмотрите поля orders: id (первичный ключ), status (например, new), created_at (дата создания; по желанию). Пользователь и регистрация для этой лабораторной не нужны.
3. Предусмотрите поля order_items: id (первичный ключ), order_id (внешний ключ на orders.id), product_id (внешний ключ на products.id), quantity (количество, больше нуля), price (цена за единицу, взятая из products.price).
4. Укажите связи в Prisma Schema и подготовьте таблицы согласованным с преподавателем способом, не используя migrate reset. Для успешного опыта используйте существующие продукты id = 1 и id = 2; для опыта с ошибкой замените id второй позиции на отсутствующий 999999.
5. В отчёт включите небольшую ER-диаграмму связей orders → order_items → products.

9.2. Что именно входит в одну транзакцию

Транзакция начинается непосредственно перед созданием заказа и заканчивается только после успешного добавления ВСЕХ его позиций. Внутри prisma.$transaction(async (tx) => { ... }) ВСЕ обращения к базе должны использовать tx, а не prisma. Если появляется ошибка, callback должен завершиться исключением, чтобы Prisma выполнила ROLLBACK.

Шаг	Операция внутри транзакции	Если возникла ошибка
1	Создать запись в orders со статусом new; получить order.id	Созданная запись будет отменена при последующем ROLLBACK
2	Найти продукт 1 (Laptop Pro, id = 1) через tx.products.findUnique и	Выбросить ошибку (throw), не переходить к следующему шагу

Шаг	Операция внутри транзакции	Если возникла ошибка
	проверить наличие	
3	Создать первую запись в order_items с order_id = order.id, product_id, quantity и ценой из products	При последующей ошибке первая позиция тоже отменяется
4	Найти продукт 2 (Wireless Mouse, id = 2) и создать вторую позицию; для демонстрации ROLLBACK использовать отсутствующий id = 999999	При ошибке второй позиции отменяются и заказ, и первая позиция
5	Успешно завершить callback, вернуть результат	Prisma фиксирует все записи (COMMIT)

Где проходит граница транзакции

В этой учебной задаче создание orders и всех order_items, включая проверки товаров, выполняется внутри ОДНОЙ транзакции. НЕЛЬЗЯ сначала создать orders отдельным prisma.orders.create(), а потом начать транзакцию только для позиций: при сбое останется пустой заказ. НЕЛЬЗЯ перехватывать ошибку внутри callback и продолжать без throw — в этом случае callback может завершиться успешно и изменения зафиксируются.

9.3. Алгоритм для самостоятельного программирования

6. Создайте отдельный файл order-transaction.js и подключите существующий Prisma Client.
7. Сформируйте массив из двух позиций: {productId: 1, quantity: 1} и {productId: 2, quantity: 2}. Для проверки ROLLBACK поменяйте только productId второй позиции на 999999.
8. Вызовите prisma.$transaction(async (tx) => { ... }). В начале callback создайте заказ (orders) и запомните его id.
9. Для каждой позиции прочитайте товар через tx, убедитесь, что товар существует и in_stock = true, а quantity — положительное целое число. При нарушении условия выполните throw new Error(...).
10. Добавьте запись в order_items через tx: order_id получайте из созданного заказа, price — из найденного товара, а не из входного массива.
11. По завершении цикла верните данные заказа из callback. За пределами callback выведите сообщение об успехе; исключение обработайте через try/catch снаружи $transaction().

Каркас для заполнения студентом (не готовое решение)

```
import { prisma } from "./config/prisma.js";

async function createOrder(items) {
  return prisma.$transaction(async (tx) => {
    // 1. Создать один заказ через tx.orders.create(...)
    //    и сохранить его id.

    for (const item of items) {
      // 2. Найти товар через tx.products.findUnique(...)
```

```
      // 3. Проверить, что товар существует и in_stock = true.
      //    Проверить, что quantity > 0.
      //    Если условие не выполнено: throw new Error(...)
      // 4. Создать позицию через tx.order_items.create(...)
      //    order_id = id созданного заказа
      //    product_id = id найденного товара
      //    price = product.price, quantity = item.quantity
    }

    // 5. Вернуть созданный заказ.
    // После успешного завершения callback Prisma сделает COMMIT.
  });
}

// Сценарий А: id = 1 и id = 2 — ожидается COMMIT.
// Сценарий Б: id = 1 и id = 999999 — ожидается ROLLBACK.
// Имена tx.orders / tx.order_items зависят от ваших моделей Prisma.
```

9.4. Два обязательных испытания

Испытание	Что происходит	Что должно быть в PostgreSQL
А. id = 1 и id = 2	Создать orders, добавить два order_items	Один новый заказ и две его позиции (COMMIT)
Б. id = 1 и id = 999999	После создания заказа и первой позиции второй товар не найден: throw	Ни заказ, ни его первая позиция не остаются в базе (ROLLBACK)

Для проверки достаточно простых запросов SELECT *FROM orders; и SELECT* FROM order_items; Перед каждым испытанием запишите число строк в обеих таблицах, после — сравните. Не путайте старые записи с новым тестовым заказом.

pgAdmin: простая проверка созданных заказов

```
SELECT * FROM orders;
SELECT * FROM order_items;
```

9.5. Что нельзя включать в проверяемую транзакцию

Не требуется включать в неё установку пакетов, миграции, создание таблиц, входящий HTTP-запрос или вывод сообщения в консоль. Для этой лабораторной транзакция охватывает именно связанные операции PostgreSQL: создать orders, проверить товары, создать все order_items. После завершения транзакции можно сформировать HTTP-ответ или вывести результат в консоль.

Обязательная проверка

Скриншот сообщения «первый товар добавлен» ещё не доказывает COMMIT: действие внутри транзакции может быть позже отменено. Результат подтверждается запросами SELECT к таблицам PostgreSQL после выполнения программы.

10. Содержание отчёта

12. Тема, цель лабораторной и использованные технологии.
13. Скриншоты SELECT * FROM products после опыта без транзакции: сохранён только «LAB9 Кабель».
14. Скриншоты SELECT * FROM products после ROLLBACK (нет тестовых товаров) и после COMMIT (есть оба тестовых товара).
15. ER-диаграмма и описание структуры orders, order_items, products.
16. Схема последовательности: где начинается транзакция, какие запросы выполняются через tx, где COMMIT/ROLLBACK.
17. Собственный код создания заказа (без копирования готового решения из предыдущей лекции).
18. Скриншоты SELECT *FROM orders u SELECT* FROM order_items для двух испытаний: COMMIT и ROLLBACK.
19. Краткий вывод, зачем в магазинном приложении нужна транзакция.

11. Контрольные вопросы

20. Почему без транзакции первый товар сохраняется, хотя второй запрос завершился ошибкой?
21. Что означает COMMIT и когда он выполняется в callback $transaction()?
22. Что означает ROLLBACK и какие действия он отменяет?
23. Какие именно операции должны находиться внутри транзакции при создании заказа с двумя позициями?
24. Почему нельзя создавать orders вне транзакции, а order_items — внутри?
25. Почему все обращения к базе внутри callback должны выполняться через tx?
26. Что произойдёт, если ошибку второй позиции обработать внутри callback без throw?
27. Откуда следует брать цену товара при создании order_items?
28. Как запросом SELECT доказать, что после ошибки в базе нет частичного заказа?

12. Критерии оценивания

Критерий	Баллы
Проверка исходных категорий и товаров (1, 2), подготовка проекта	10
Опыт без транзакции, анализ частичного результата	15
Опыт с транзакцией: подтверждение ROLLBACK и COMMIT	20
Проектирование orders и order_items, ER-диаграмма	15
Самостоятельная реализация транзакции создания заказа	20
Два испытания, SQL-доказательства результатов	15
Отчёт и ответы на контрольные вопросы	5
Итого	100